819G5: Artificial Life
Coursework

Using genetic algorithm to determine parameters for Rabin fingerprint dediplication

Temir Zhumagali 22512702

16.01.2026

1. Introduction

For several areas related to data manipulation such as data deduplication, sharing or cybersecurity, identifying the same patterns of data is proved to be crucial. As reported by Csaplar (2013) the mean backup data size for medium enterprise reached 285TB. Several cloud storage services, particularly Dropbox, Google Drive, Bitcasa are using data deduplication to reduce cloud storage cost (Su et al., 2017).

Data deduplication is a commonly used method of reducing storage maintenance cost by deleting redundant data. It identifies boundaries of the pieces of the files, referred to as 'chunks', and compares them. For each chunk a unique combination of symbols, called fingerprint, is calculated via hashing (SHA256). If chunks have identical fingerprints, only one copy is stored, the others are referenced to by pointers and not being saved (Xia et al., 2016). There are several types of data deduplication, one of which is content defined chunking based on Rabin fingerprinting. It represents a fixed sized byte window of a file in terms of polynomial and divides by a predetermined irreducible polynomial over Galois field 2 (GF(2)). If a remainder of division satisfies certain predetermined conditions, the boundary of a chunk is placed and a new window will begin the next. Such an approach is more robust to insertions or delitions of bytes, allowing more effective deduplication compared to fixed sized chunking (Su et al., 2017).

In order to function properly Rabin fingerprint requires certain predetermined parameters, such as sliding window size, degree of polynomial, polynomial itself and a mask, representing condition to place boundary of a chunk. In this paper population based microbial genetic algorithms (GA) will be used to determine those parameters. GA in general are optimization algorithms based on the idea of evolution (Mathew,2012). Parameters are treated as genes in the genotype that are evolving over each generation. Evolution is represented in changing parameters and calculating the fitness of genotype. Fitness is a parameter of GA responsible for indicating the performance of a genotype. In the context of this particular work fitness means the degree of saved storage space, because it is a primary function of data deduplication. Microbial GA is characterised by properties of horizontal transmission and steady state. Horizontal transmission means copying genes from individuals from the same generation,while steady state means replacing only one individual each generation. Genotype with higher fitness would pass its genetic material to a member population with lower fitness, possibly increasing their fitness while also maintaining the possibility to find new evolutionary paths with higher fitness (Mathew,2012).

The hypothesis is that microbial GA can be more effective for choosing fingerprinting parameters and also provide valuable insights into relationships between parameters.

This paper aims to answer the question: to what degree microbial GA is effective for choosing Rabin fingerprint parameters for data deduplication.

To address this question, a python code for deduplication was written. Rabin fingerprint parameters were optimized with microbial GA.

2. Methodology

Rabin fingerprint

Rabin fingerprint is known for relatively quick identification of duplicate data. It is based on irreducible polynomials with coefficients in GF(2). For example let $S = [a_1, a_2,..., a_n]$ be a binary string, with either 1 or 0. Then

$$S(t) = a_1 t^{n-1} + a_2 t^{n-2} +... + a_{n-1} t + a_n$$

is polynomial with degree n-1 representing string S (Su et al., 2017).

$$P(t) = b_1 t^n + b_2 t^{n-1} +... + b_k t + b_{k+1}$$

$P(t)$ is a predetermined irreducible polynomial with degree of k, that is one of the parameters to evolve. The Rabin fingerprint of S is:

$$r(t) = S(t) \, mod \, p(t)$$

It is crucial to notice that mod over GF(2) is different from finding the remainder of other polynomials. In order to find the remainder it is required to use formula $A = A^{B<<(degree(A)-degree(B))}$ until degree of A will not be less than degree of B. What is left of A is the remainder, thus $r(t)$=A (DG, 2022).

Let $[X_1, X_2,..., X_z,..., X_{z+1},...]$ be a string of bytes of a sliding window with the width of z. Assuming that the starting position is X_i Rabin fingerprint value of a string $[X_i, X_{i+1},..., X_{z-i+1}]$ in the window is

$$r_i(t) = \left(\sum_{j=1}^{\omega} X_{i+j-1}(t) \, t^{8\omega - j} \right) \bmod p(t)$$

Each $r_i(t)$ is Rabin fingerprint value of a chunk (Mathew,2012). Each chunk was stored in a separate folder and the names were assigned by hash value. In order to prove the correctness of working of the algorithm, the restore_file method was created. The method received the list containing all hashes of chunks, and combined chunks by order into one file.

Microbial GA

Rabin fingerprint requires an irreducible polynomial, which is difficult to calculate. Thus it was decided to use primitive polynomials instead, since they by definition are also irreducible. The table containing primitive polynomials was made by Moon (2005).

Lu et al. (2019) created a table of possible reasonable Rabin fingerprint parameters. Possible values for each were adjusted accordingly, excluding cut value. All data is presented at Table 1.

The pseudocode for Microbial GA is provided in Figure 1. Microbial GA is used to evolve the parameters of the Rabin fingerprint, including the choice of the primitive polynomial degree and window size.

Label	Sliding Window Size (W) (bits)	Degree of Polynomial (P)	Mask Size (M) (bits)	Cut Value (C)	Total Number of chunks	Average chunk length (bytes)	Std Dev (bytes)	Median chunk length (bytes)
$2^{18} = 262,144$, $2^{16} = 65,536$, $2^{12} = 4,096$, $2^{8} = 256$, $2^{4} = 16$								
W18, P17, M18, C1	18	17	18	1	20,310	319,541	484,884	127,204
W18, P17, M16, C1	18	17	16	1	84,255	77,026	115,385	34,493
W18, P17, M12, C1	18	17	12	1	1,469,749	4,415	6,143	2,267
W18, P17, M8, C1	18	17	8	1	23,441,264	276	344	163
W18, P17, M4, C1	18	17	4	1	385,288,489	16	21	11
W16, P15, M16, C1	16	15	16	1	89,653	72,389	121,494	22,385
W16, P15, M12, C1	16	15	12	1	1,916,620	3,386	4,694	1,897
W16, P15, M8, C1	16	15	8	1	23,806,675	272	332	169
W16, P15, M4, C1	16	15	4	1	389,630,482	16	21	11
W16, P11, M12, C1	16	11	12	1	1,474,937	4,400	6,515	2,079
W16, P7, M8, C1	16	7	8	1	23,946,583	271	319	167
W16, P7, M4, C1	16	7	4	1	382,869,747	16	21	11
W12, P11, M12, C1	12	11	12	1	1,415,327	4,585	7,318	1,823
W12, P7, M8, C1	12	7	8	1	22,835,958	284	381	154
W12, P7, M4, C1	12	7	4	1	380,006,136	17	23	11
W8, P7, M8, C1	8	7	8	1	29,345,409	221	371	109
W8, P7, M4, C1	8	7	4	1	385,563,093	16	22	11
W4, P7, M4, C1	4	7	4	1	361,518,831	17	25	12
W4, P7, M2, C1	4	7	2	1	1,618,304,242	4	7	3
Alternate cut values								
W16, P15, M16, C0	16	15	16	0	301,452,622	21	1,027	1
W16, P15, M16, C3	16	15	16	3	90,567	71,658	114,856	31,122
W16, P15, M16, C7	16	15	16	7	106,522	60,925	111,375	13,986

Table 1. Reasonable parameters for Rabin fingerprint by Lu et al. (2019).

```
Algorithm 1 Microbial genetic algorithm
1: function MICRO_GA(fitness_target, pop_size, p_mutation,microbial_max, min_size, max_size, D)
2:     pop_genotypes ← empty list
3:     pop_fitnesses ← empty list
4:     best_fitness_history ← empty list
5:     microbial_current ← 0
6:     fitness_best ← 0
7:     for i = 0 to (pop_size − 1) do
8:         genotype ← RANDOM_GENOTYPE(min_size, max_size)
9:         append genotype to pop_genotypes
10:        append FITNESS_FUNCTION(genotype) to pop_fitnesses
11:    end for
12:    while fitness_best ≤ fitness_target and microbial_current < microbial_max do
13:        winner_index, loser_index ←MICROBIAL_SELECTION(pop_genotypes, pop_fitnesses, D)
14:        loser_copy ← UNIFORM_CROSSOVER(pop_genotypes, pop_genotypes, probability_cross, length)
15:        pop_genotypes[loser_index] ←MUTATE_GENOTYPE(loser_copy, p_mutation)
16:        pop_fitnesses[loser_index] ←FITNESS_FUNCTION(pop_genotypes[loser_index])
17:        microbial_current ← microbial_current + 1
18:        fitness_best ← MAX(pop_fitnesses)
19:        append fitness_best to best_fitness_history
20:    end while
21:    return pop_genotypes, pop_fitnesses, best_fitness_history
22: end function
```

Figure 1. Pseudocode for microbial GA

For testing algorithms, a file with 30 percent of duplicated data was created. Duplication was performed by creating identical blocks of data with fixed size, in this case 64 bytes.

3. Results and Discussion

Performance of the model was measured by fitness value (highest possible is 30), which was indicating percentage to which folder with chunks was smaller than the file. Population size 10 and number of microbial tournaments =200. The acceptance function was netcrawler. Parameters for evolution are: degree of polynomial, irreducible polynomial, size of the sliding window, degree of mask, minimal and maximal size of a chunk.

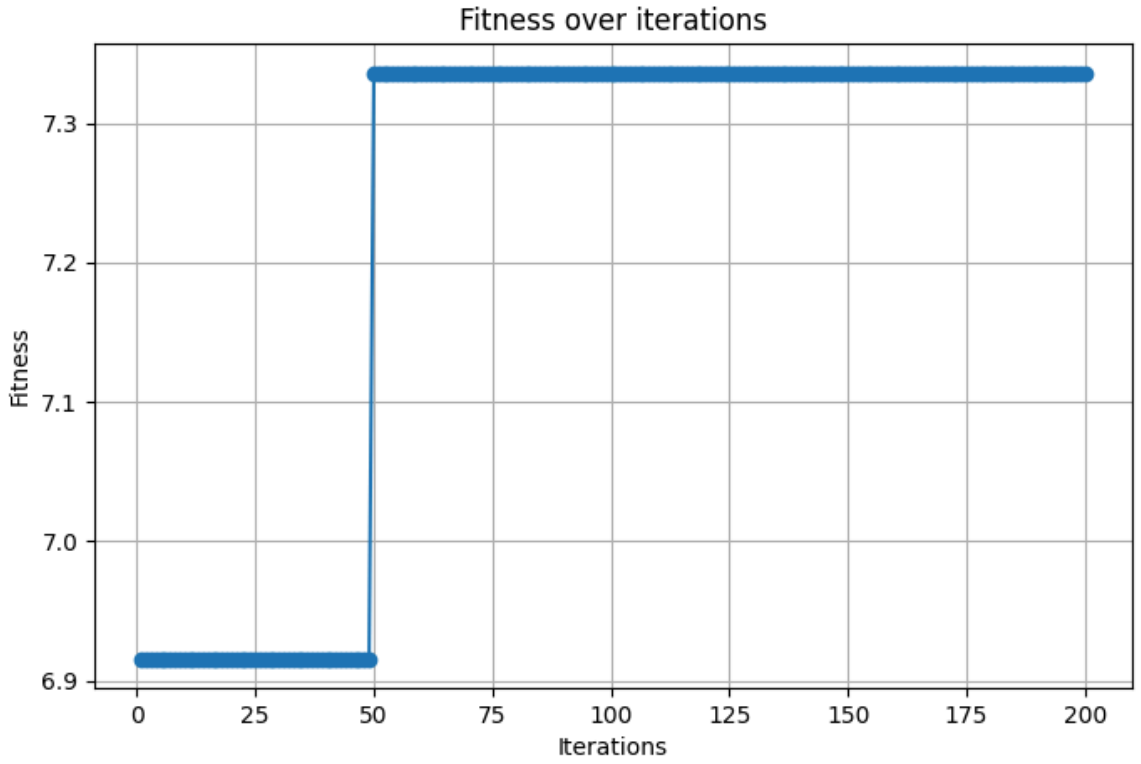


Figure 2. Fitness over iterations with size of the file 150kb and percent of duplications 30.

Figure 2 displays results of the first experiment. The fitness achieves a point of convergence at tournament (iteration) 50 and remains the same. The lack of further fitness improvement after convergence suggests a neutral fitness landscape, where multiple genotypes map to similar chunking parameters.

The results of tournament selection in Table 3 suggest that genotypes quickly converge into the same genotype with highest fitness value, not able to break through neutrality. While microbial tournament selection approaches differently, applying ‘trivial geography’. One genotype is compared with its neighbor at a distance close to the value of D,

which is 5. By doing so microbial tournament selection keeps genotype divergent, possibly increasing probability to find a global optima. Although in this experiment it was not achieved.

Degree of polynomial	Polynomial	Degree of sliding window	Degree of mask	min chunk size	max chunk size	Fitness
11	3085	8	16	64	8252	6.99895
11	4143	12	4	64	8004	6.29203
11	2189	8	4	64	8624	6.63052
2	7	16	4	312	7756	0.00000
11	2579	4	18	64	7756	7.33614
10	1305	8	12	312	7632	0.00000
8	487	8	18	436	6888	0.00000
11	4215	12	18	64	7012	6.14687
9	875	8	12	188	6764	0.12498

Table 1. Results from using microbial tournament selection

Candidate	Degree of polynomial	Polynomial	Degree of sliding window	Degree of mask	min chunk size	max chunk size	Fitness
Original	11	2053	18	486	1024	6144	0.00000
Best	11	2579	4	18	64	7756	7.33614

Table 2. Comparing initial randomly generated

Degree of polynomial	Polynomial	Degree of sliding window	Degree of mask	min chunk size	max chunk size	Fitness
5	55	4	18	64	6392	6.38647
5	55	4	4	64	6888	6.33374
8	351	4	18	64	8004	6.41707
7	131	4	2	312	8996	0.00000
7	131	4	8	64	6268	6.33309
7	131	4	18	64	8128	6.45808
6	103	4	2	64	7260	6.33504
7	131	4	18	64	8500	6.45808
7	229	8	18	64	8376	6.37736

Table 3. Results of using tournament selection

Several candidates achieved 0 fitness, indicating incompatible combinations of parameters. This highlights the sensitivity of Rabin fingerprint.

Table 2 represents comparison of initial parameters and final best genotype. This results displays that choosing random parameters for Rabin fingerprint is not an optimal choice.

The results suggest that the degree of sliding window, minimal size of the chunk and degree of mask are the most important parameters. It is reasonable because the idea of Rabin fingerprint is to identify chunks of redundant data and those three parameters are the most important in identifying the mean size of the chunks.

4. Conclusion

This experiment proved that while it is possible to find better values for Rabin fingerprint parameters using microbial GA, the degree is not impressive. While the first randomly generated candidate achieved only fitness 0, following ones quickly reached the point in which the difference between fitnesses is not significant. Despite that, using GA provided further evidence of sensitivity of Rabin fingerprint to changing in parameters.

Future work will include experiments with larger populations, different file sizes as well as alternative acceptance functions.

References

DG. (2022, February 14). *Polynomial division in GF(2) – finding remainder* [Video]. YouTube. https://www.youtube.com/watch?v=ApIFldw-Hpw

Csaplar, D. (2013). *Who is adopting the public cloud faster? North America or Europe?* Aberdeen Group. Technical report.

Harvey, I. (2011). The microbial genetic algorithm. In *Lecture Notes in Computer Science* (pp. 126–133). Springer. https://doi.org/10.1007/978-3-642-21314-4_16

Lu, P., Randall, O., & McDonald, E. (2019). An empirical study of Rabin fingerprinting parameters. In *Proceedings of the IEEE International Conference on Big Data* (pp. 3686–3691). https://doi.org/10.1109/BigData47090.2019.9006560

Mathew, T. V. (2012). *Genetic algorithm*. Report submitted at IIT Bombay.

Moon, T. K. (2005). *Error correction coding: Mathematical methods and algorithms*. Wiley.

Su, H., Zheng, D., & Zhang, Y. (2017). An efficient and secure deduplication scheme based on Rabin fingerprinting in cloud storage. In *Proceedings of the IEEE Conference on Computer Science and Education (CSE-EUC)* (pp. 833–836). https://doi.org/10.1109/CSE-EUC.2017.166

Xia, W., Jiang, H., Feng, D., Douglis, F., Shilane, P., Hua, Y., Fu, M., Zhang, Y., & Zhou, Y. (2016). A comprehensive study of the past, present, and future of data deduplication. *Proceedings of the IEEE, 104*(9), 1681–1710. https://doi.org/10.1109/JPROC.2016.2571298

819G5: Artificial Life
Coursework